

Setul de instrucțiuni al familiei de procesoare Intel x86

Scopul lucrării

În cadrul acestei lucrări se prezintă o primă parte din instrucțiunile limbajului de asamblare al procesoarelor Intel x86 (varianta pe 32 de biti). Restul instrucțiunilor vor fi prezentate în lucrarea următoare. În prima parte a lucrării se prezintă sintaxa generală a unei instrucțiuni de asamblare și câteva reguli de scriere a programelor. În partea practică a lucrării se vor scrie programe cu instrucțiunile prezentate și se va analiza efectul execuției acestora.

Prezentarea instrucțiunilor

Un limbaj de asamblare conține instrucțiuni corespunzătoare unor operații simple care sunt direct interpretate și executate de procesor. Fiecărei instrucțiuni din limbajul de asamblare îi corespunde în mod strict un singur cod executabil. În contrast, unei instrucțiuni dintr-un limbaj de nivel înalt (ex: C, Pascal, etc.) îi corespunde o secvență de coduri (instrucțiuni în cod mașină). Un anumit limbaj de asamblare este specific pentru un anumit procesor sau eventual pentru o familie de procesoare. Instrucțiunile sunt în directă corelație cu structura internă a procesorului. Un programator în limbaj de asamblare trebuie să cunoască această structură precum și tipurile de operații permise de structura respectivă.

Un program în asamblare scris pentru o anumită arhitectură de procesor nu este compatibil cu un alt tip de procesor. Pentru implementarea unei aplicații pe un alt procesor programul trebuie rescris. În schimb programele scrise în limbaj de asamblare sunt în general mai eficiente atât în ceea ce privește timpul de execuție cât și spațiul de memorie ocupat de program. De asemenea, programarea în limbaj de asamblare dă o mai mare flexibilitate și libertate în utilizarea resurselor unui calculator. Cu toate acestea astăzi utilizarea limbajului de asamblare este mai puțin frecventă deoarece eficiența procesului de programare este mai scăzută, există puține structuri de program și de date care să ușureze munca programatorului, iar programatorul trebuie să cunoască structura procesorului pentru care scrie aplicația. În plus programele nu sunt portabile, adică nu rulează și pe alte procesoare.

Un program scris în limbaj de asamblare conține instrucțiuni și directive. Instrucțiunile sunt traduse în coduri executate de procesor; ele se regăsesc în programul executabil generat în urma compilării și a editării de legături. Directivele sunt construcții de limbaj ajutătoare care se utilizează în diferite scopuri (ex: declararea variabilelor, demarcarea segmentelor și a procedurilor, etc.) și au rol în special în fazele de compilare și editare de legături. O directivă nu se traduce printr-un cod executabil și în consecință NU se execută de către procesor.

Sintaxa unei instrucțiuni în limbaj de asamblare

O instrucțiune ocupă o linie de program și se compune din mai multe câmpuri, după cum urmează (parantezele drepte indică faptul că un anumit câmp poate să lipsească):

[<eticheta>:] [<mnemonica> [<parametru_1> [<parametru_2>]] [<comentariu>]

- <eticheta> - este un nume simbolic (identificator) dat unei locații de memorie care conține instrucțiunea care urmează; scopul unei etichete este de a indica locul în care trebuie să se facă un salt în urma executării unei instrucțiuni de salt; eticheta poate fi o combinație de litere, cifre și anumite semne speciale (ex: _), cu restricția ca prima cifră să fie o literă

- <mnemonica> - este o combinație de litere care simbolizează o anumită instrucțiune (ex: add pentru adunare, mov pentru transfer, etc.); denumirile de instrucțiuni sunt cuvinte rezervate și nu pot fi utilizate în alte scopuri
- <parametru_1> - este primul operand al unei instrucțiuni și în același timp și destinația rezultatului; primul parametru poate fi un registru, o adresă, sau o expresie care generează o adresă de operand; adresa operandului se poate exprima și printr-un nume simbolic (numele dat unei variabile)
- <parametru_2> - este al doilea operand al unei instrucțiuni; acesta poate fi oricare din variantele prezentate la primul operand și în plus poate fi și o constantă
- <comentariu> - este un text explicativ care arată intențiile programatorului și efectul scontat în urma execuției instrucțiunii; având în vedere că programele scrise în limbaj de asamblare sunt mai greu de interpretat se impune aproape în mod obligatoriu utilizarea de comentarii; textul comentariului este ignorat de compilator; comentariul se considera până la sfârșitul liniei curente

Într-o linie de program nu toate câmpurile sunt obligatorii: poate să lipsească eticheta, parametrii, comentariul sau chiar instrucțiunea. Unele instrucțiuni nu necesită nici un parametru, altele au nevoie de unul sau doi parametri. În principiu primul parametru este destinația, iar al doilea este sursa.

Constantele numerice care apar în program se pot exprima în zecimal (modul implicit), în hexazecimal (constante terminate cu litera 'h') sau în binar (constante terminate cu litera 'b'). Constantele alfanumerice (coduri ASCII) se exprimă prin litere între apostrof sau text între ghilimele.

Clase de instrucțiuni

În această prezentare nu se va face o prezentare exhaustivă a tuturor instrucțiunilor cu toate detaliile lor de execuție. Se vor prezenta acele instrucțiuni care se utilizează mai des și au importanță din punct de vedere al structurii și al posibilităților procesorului. Pentru alte detalii se pot consulta documentații complete referitoare la setul de instrucțiuni ISA x86 (ex: cursul AoA – The Art of Assembly Language, accesibil pe Internet).

Instrucțiuni de transfer

Instrucțiunile de transfer realizează transferul de date între registre, între un registru și o locație de memorie sau o constantă se încarcă într-un registru sau locație de memorie. **Transferurile de tip memorie-memorie nu sunt permise** (cu excepția instrucțiunilor pe șiruri). La fel nu sunt permise transferurile directe între două registre segment. Ambii parametri ai unui transfer trebuie să aibă aceeași lungime (număr de biți).

Instrucțiunea MOV

Este cea mai utilizată instrucțiune de transfer. Sintaxa ei este:

[<eticheta>:] MOV <parametru_1>, <parametru_2> [;<comentariu>]

unde:

<parametru_1> = <registru>| <reg_segment>|<adresa_offset>|<nume_variabilă>|<expresie>

<parametru_2> = <parametru_1>|<constantă>

<registru> = EAX|EBX|....ESP|AX|BX|....SP|AH|AL|....DL

<expresie> = [[<registru_index>][+<registru_bază>][+<deplasament>]]

; aici parantezele drepte marcate cu bold sunt necesare

<registru_index> = SI | DI | ESI | EDI
<registru_bază> = BX | BP | EBX | EBP
<deplasament> = <constantă>

Exemple:

```
mov ax,bx          et1:  mov ah, [si+100h]
mov cl, 12h         mov al, 'A'
mov dx, var16       mov si, 1234h
mov var32,eax       sf:   mov [si+bx+30h], dx
mov ds, ax          mov bx, cs
```

Exemple de erori de sintaxă:

```
mov ax, cl          ; operanzi inegali ca lungime
mov var1, var2      ; ambii operanzi sunt locații de memorie
mov al, 1234h       ; dimensiunea constantei este mai mare decât cea a registrului
mov ds, es          ; două registre segment
```

Instrucțiunea LEA

Aceste instrucțiuni permit încărcarea într-un registru a unei adrese de variabile. Prima instrucțiune LEA ("load effective address") încarcă în registrul exprimat ca prim parametru adresa liniară a variabilei din parametrul 2.

LEA <parametru_1>, <parametru_2>

Exemple:

```
lea esi, var1       ; ESI<= offset(var1)
lea edi, [ebx+100]  ; EDI<= EBX+100
```

Instrucțiunea LEA este echivalenta (are același efect) cu următoarea instrucțiune:

MOV <registru>, **offset** <memorie>

Instrucțiunea XCHG

Această instrucțiune schimbă între ele conținutul celor doi operanzi.

XCHG <parametru_1>, <parametru_2>

Atenție: parametrul 2 nu poate fi o constantă.

Exemple:

```
xchg al, bh
xchg ax, bx
```

Instrucțiunile PUSH și POP

Cele două instrucțiuni operează în mod implicit cu vârful stivei. Instrucțiunea PUSH pune un operand pe stivă, iar POP extrage o valoare de pe stivă și o depune într-un operand. În ambele cazuri registrul indicator de stivă (ESP) se modifică corespunzător (prin decrementare și respectiv incrementare) astfel încât registrul ESP să indice poziția curentă a vârfului de stivă. Sintaxa instrucțiunilor este:

PUSH <parametru_1>

POP <parametru_1>

Operandul trebuie sa fie o valoare pe 16 sau 32 biți, dar **este recomandat sa se folosească doar valori pe 32 de biți**. Aceste instrucțiuni sunt utile pentru salvarea temporară și refacerea conținutului unor registre. Aceste operații sunt necesare mai ales la apelul de rutine și la revenirea din rutine. În cazul introducerii în stivă, prima operație care se realizează este decrementarea indicatorului de stivă ESP cu 2 (la introducerea unei valori pe 16 biți) sau 4 (la introducerea unei valori pe 32 biți), urmată de memorarea operandului conform acestui indicator. În cazul extragerii din stivă prima operație care se realizează este citirea operandului conform indicatorului de stivă urmată de incrementarea cu 2 sau 4 a indicatorului.

Exemple:

push eax	push var16
pop bx	pop var32

Instrucțiuni de transfer pentru indicatorii de condiție

În setul de instrucțiuni al microprocesorului I8086 există instrucțiuni pentru încărcarea și memorarea indicatorilor de condiție. Sintaxa este următoarea:

LAHF
SAHF
PUSHF
POPF

Octetul mai puțin semnificativ al registrului indicatorilor de condiție poate fi încărcat în registrul ah folosind instrucțiunea LAHF, respectiv poate fi înscris cu conținutul registrului ah folosind instrucțiunea SAHF. Structura octetului care se transferă este următoarea :

bitul	7	6	5	4	3	2	1	0
	SF	ZF	...	AF	...	PF	...	CF

Dacă se dorește salvarea sau refacerea întregului registru al indicatorilor de condiție se folosesc instrucțiunile PUSHF și POPF. Structura cuvântului care se transferă în acest caz este următoarea:

Bitul	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
..					OF	DF	IF	TF	SF	ZF	..	AF	..	PF	..	CF

Instrucțiuni aritmetice

Aceste instrucțiuni efectuează cele patru operații aritmetice de bază: adunare, scădere, înmulțire și împărțire. Rezultatul acestor instrucțiuni afectează starea indicatorilor de condiție.

Instrucțiunile ADD și ADC

Aceste instrucțiuni efectuează operația de adunare a doi operanzi, rezultatul plasându-se în primul operand. A doua instrucțiune ADC (ADD with carry) în plus adună și conținutul indicatorului de transport CF. Această instrucțiune este utilă pentru implementarea unor adunări în care operanzii sunt mai lungi de 32 de biți.

ADD <parametru_1>,<parametru_2>
ADC <parametru_1>,<parametru_2>

Exemple:

add ax, 1234h
add bx, ax
adc dx, var16

Instrucțiunile SUB și SBB

Aceste instrucțiuni implementează operația de scădere. A doua instrucțiune, SBB (Subtract with borrow) scade și conținutul indicatorului CF, folosit în acest caz pe post de bit de împrumut. Ca și ADC, SBB se folosește pentru operanzi de lungime mai mare.

SUB <parametru_1>,<parametru_2>
SBB <parametru_1>,<parametru_2>

Instrucțiunile MUL și IMUL

Aceste instrucțiuni efectuează operația de înmulțire, MUL pentru întregi fără semn și IMUL pentru întregi cu semn. De remarcat că la operațiile de înmulțire și împărțire trebuie să se țină cont de forma de reprezentare a numerelor (cu semn sau fără semn), pe când la adunare și scădere acest lucru nu este necesar. Pentru a evita dese depășiri de capacitate s-a decis ca rezultatul operației de înmulțire să se păstreze pe o lungime dublă față de lungimea operanzilor. Astfel dacă operanzii sunt pe octet rezultatul este pe cuvânt, iar dacă operanzi sunt pe cuvânt rezultatul este pe dublu-cuvânt. De asemenea se impune ca primul operand și implicit și rezultatul să se păstreze în registrul acumulator. De aceea primul operand nu se mai specifică.

MUL <operand 1>
IMUL <operand 1>
IMUL <operand 1>, <operand 2>
IMUL <operand 1>, <operand 2>, <valoare imediata>

Exemple:

mul dh ; AX<=AL*DH
mul bx ; DX:AX<= AX*BX DX este extensia registrului acumulator AX
imul var8 ; AX<=AL*var8

Instrucțiunea MUL are un singur operand explicit, ceilalți fiind implicați. În funcție de dimensiunea operandului explicit, înmulțirea are loc astfel:

MUL <operand_8biti>
Rezultat înmulțire: AX=AL*<operand_8biti>
MUL <operand_16biti>
Rezultat înmulțire: DX:AX=AX*<operand_16biti>
MUL <operand_32biti>
Rezultat înmulțire: EDX:EAX=EAX*<operand_32biti>

Aceleași reguli se aplică și la instrucțiunea IMUL cu 1 operand. La instrucțiunea IMUL cu 2 operanzi, rezultatul înmulțirii celor 2 operanzi este păstrat în primul operand. În

cazul utilizării instrucțiunii IMUL cu 3 operanzi, primul operand păstrează rezultatul înmulțirii între al doilea operand și valoarea imediată.

Instrucțiunile DIV și IDIV

Aceste instrucțiuni efectuează operația de împărțire pe întregi fără semn și respectiv cu semn. Pentru a crește plaja de operare se consideră că primul operand, care în mod obligatoriu trebuie să fie în acumulator, are o lungime dublă față de al doilea operand. Primul operand nu se specifică.

DIV <operand>
IDIV <operand>

Exemple:

div cl ; AL<=AX/CL și AH<=AX mod CL (adică restul împărțirii)
div bx ; AX<= (DX:AX)/BX și DX<=(DX:AX) mod BX

Instrucțiunile DIV și IDIV au un singur operand explicit, ceilalți fiind implicați. În funcție de dimensiunea operandului explicit, împărțirea are loc astfel:

DIV <operand_8biti>
Rezultat împărțire: AL=AX/<operand_8biti>; AH=AX mod <operand_8biti>
DIV <operand_16biti>
Rezultat împărțire: AX=DX:AX/<operand_16biti>; DX=DX:AX mod <operand_16biti>
DIV <operand_32biti>
Rezultat împărțire: EAX=EDX:EAX/<operand_32biti>; EDX=EAX mod <operand_32biti>

Instrucțiunile INC și DEC

Aceste instrucțiuni realizează incrementarea și respectiv decrementarea cu o unitate a operandului. Aceste instrucțiuni sunt eficiente ca lungime și ca viteză. Ele se folosesc pentru contorizare și pentru parcurgerea unor șiruri prin incrementarea sau decrementarea adreselor.

INC <parametru>
DEC <parametru>

Exemple:

inc si ; SI<=SI+1
dec cx ; CX<=CX-1

Instrucțiunea CMP

Această instrucțiune compară cei doi operanzi prin scăderea lor. Rezultatul scăderii nu se memorează. Instrucțiunea are efect numai asupra următorilor indicatori de condiție: ZF, SF, OF, CF. Valorile indicatorilor de condiție pot fi apoi interpretate în mod diferit dacă valorile comparate au fost cu semn sau fără semn. Această instrucțiune precede de obicei o instrucțiune de salt condiționat. printr-o combinație de instrucțiune de comparare și o instrucțiune de salt se pot verifica relații de egalitate, mai mare, mai mic, mai mare sau egal, etc.

CMP <parametru_1>, <parametru_2>

Exemplu:

```
cmp ax, 50h
```

Instrucțiuni logice

Aceste instrucțiuni implementează operațiile de bază ale logicii booleene. Operațiile logice se efectuează la nivel de bit, adică se combină printr-o operație logică fiecare bit al operandului 1 cu bitul corespunzător din operandul al doilea. Rezultatul se generează în primul operand.

Instrucțiunile AND, OR, NOT și XOR

Aceste instrucțiuni implementează cele patru operații de bază: ȘI, SAU, Negație și SAU-Exclusiv.

```
AND <parametru_1>,<parametru_2>
```

```
OR <parametru_1>, <parametru_2>
```

```
NOT <parametru_1>
```

```
XOR <parametru_1>,<parametru_2>
```

Exemple:

```
and al, 0fh
```

```
or bx, 0000111100001111b
```

```
and al,ch
```

```
xor eax,eax ; șterge conținutul lui eax
```

Instrucțiunea TEST

Această instrucțiune efectuează operația ȘI logic fără a memora rezultatul. Scopul operației este de a modifica indicatorii de condiție. Instrucțiunea evită distrugerea conținutului primului operand.

```
TEST <parametru_1>,<parametru_2>
```

Exemple:

```
test al, 00010000b ; se verifică dacă bitul D4 din al este zero sau nu
```

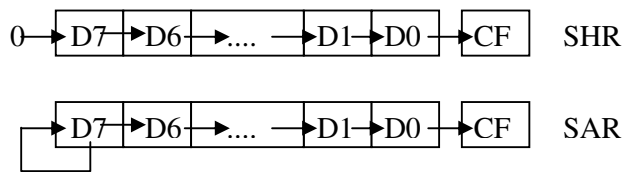
```
test bl, 0fh ;se verifica dacă prima cifră hexazecimală din bl este 0
```

Instrucțiuni de deplasare și de rotire

Instrucțiunile SHL, (SAL), SHR și SAR

Aceste instrucțiuni realizează deplasarea (eng. shift) la stânga și respectiv la dreapta a conținutului unui operand. La deplasarea "logică" (SHL, SHR) biții se copiază în locațiile învecinate (la stânga sau la dreapta), iar pe locurile rămase libere se înscrie 0 logic. La deplasarea "aritmetică" (SAL, SAR) se consideră că operandul conține un număr cu semn, iar prin deplasare la stânga și la dreapta se obține o multiplicare și respectiv o divizare cu puteri ale lui doi (ex: o deplasare la stânga cu 2 poziții binare este echivalent cu o înmulțire cu 4). La deplasarea la dreapta se dorește menținerea semnului operandului, de aceea bitul mai semnificativ (semnul) se menține și după deplasare. La deplasarea la stânga acest lucru nu este necesar, de aceea instrucțiunile SHL și SAL reprezintă aceeași instrucțiune.

În figura de mai jos s-a reprezentat o deplasare logică și o deplasare aritmetică la dreapta. Se observă că bitul care iese din operand este înscris în indicatorul de transport CF



Formatul instrucțiunilor:

```
SHL    <parametru_1>, <parametru_2>
SAL    <parametru_1>, <parametru_2>
SHR    <parametru_1>, <parametru_2>
SAR    <parametru_1>, <parametru_2>
```

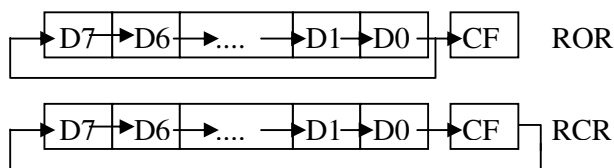
Primul parametru este în concordanță cu definițiile anterioare; al doilea parametru specifică numărul de poziții binare cu care se face deplasare; acest parametru poate fi 1 sau dacă numărul de pași este mai mare atunci se indică prin registrul CL. Incepand cu procesorul 80386 se acceptă și forma în care constanta este diferită de 1.

Exemple:

```
shl    eax, 1
sar    ebx, 3
shr    var, cl
```

Instrucțiunile de rotire ROR, ROL, RCR, RCL

Aceste instrucțiuni realizează rotirea la dreapta sau la stânga a operandului, cu un număr de poziții binare. Diferența față de instrucțiunile anterioare de deplasare constă în faptul că în poziția eliberată prin deplasare se introduce bitul care iese din operand. Rotirea se poate face cu implicarea indicatorului de transport (CF) în procesul de rotație (RCR, RCL) sau fără (ROR, ROL). În ambele cazuri bitul care iese din operand se regăsește în indicatorul de transport CF. Figura de mai jos indică cele două moduri de rotație pentru o rotație la dreapta.



Formatul instrucțiunilor:

```
ROR    <parametru_1>, <parametru_2>
ROL    <parametru_1>, <parametru_2>
RCR    <parametru_1>, <parametru_2>
RCL    <parametru_1>, <parametru_2>
```

Referitor la parametri, se aplică aceleași observații ca și la instrucțiunile de deplasare.

Instrucțiuni de intrare/ieșire

Aceste instrucțiuni se utilizează pentru efectuarea transferurilor cu registrele (porturile) interfețelor de intrare/ieșire. Trebuie remarcat faptul că la procesoarele Intel acestea sunt singurele instrucțiuni care operează cu porturi.

Instrucțiunile IN și OUT

Instrucțiunea IN se folosește pentru citirea unui port de intrare, iar instrucțiunea OUT pentru scrierea unui port de ieșire. Sintaxa instrucțiunilor este:

IN <acumulator>, <adresă_port>

OUT <adresă_port>, <acumulator>

unde:

<acumulator> - registrul EAX/AX/AL pentru transfer pe 32/16/8 biți

<adresa_port> - o adresă exprimabilă pe 8 biți sau registrul DX

Se observă că dacă adresa portului este mai mare decât 255 atunci adresa portului se transmite prin registrul DX.

Exemple:

in al, 20h

mov dx, adresa_port

out dx, ax

Instrucțiuni speciale

În această categorie s-au inclus acele instrucțiuni care au efect asupra modului de funcționare al procesorului.

Instrucțiunile CLC, STC, CMC

Aceste instrucțiuni modifică starea indicatorului CF de transport. Aceste instrucțiuni au următoarele efecte:

CLC șterge (eng. clear) indicatorul, CF=0

STC setează indicatorul, CF=1

CMC inversează starea indicatorului, CF=NOT CF

Instrucțiunile CLI și STI

Aceste instrucțiuni șterg și respectiv setează indicatorul de întrerupere IF. În starea setată (IF=1) procesorul detectează întreruperile mascabile, iar în starea inversă blochează toate întreruperile mascabile.

Instrucțiunile CLD și STD

Aceste instrucțiuni modifică starea indicatorului de direcție DF. Prin acest indicator se controlează modul de parcurgere a șirurilor la operațiile pe șiruri: prin incrementare (DF=0) sau prin decrementare (DF=1).

Instrucțiunea CUID

Această instrucțiune permite identificarea tipului de procesor pe care rupeaza programul.

Mersul lucrării

1. Discuții legate de problemele întâmpinate în înțelegerea documentației scrise.

2. Problema rezolvată în s3ex1.asm

Sa se implementeze în limbaj de asamblare expresia de mai jos folosind instrucțiuni de deplasare:

$$EAX = 7 * EAX - 2 * EBX - EBX / 8$$

- a. Se va compila s3ex1.asm
- b. Se va executa în Ollydbg
- c. Se vor urmări schimbările care au loc la nivelul regiștrilor și a flagurilor

3. Sa se implementeze în limbaj de asamblare expresia de la punctul 2 folosind instrucțiuni aritmetice.

4. Problema rezolvată în s3ex2.asm

Sa se scrie un program în limbaj de asamblare care generează un întreg reprezentabil pe octet și îl pune în locația de memorie REZ după formula:

$$REZ = AL * NUM1 + (NUM2 * AL + BL)$$

REZ, NUM1 și NUM2 sunt valori reprezentate pe octet aflate în memorie.

- a. Se va compila s3ex2.asm
- b. Se va executa în Ollydbg
- c. Se vor urmări schimbările care au loc la nivelul regiștrilor, memoriei și a flagurilor

5. Sa se scrie un program în limbaj de asamblare care calculează un întreg reprezentabil pe cuvânt și îl pune în locația de memorie REZ după formula:

$$REZ = AX * NUM1 + (NUM2 * AX + BX)$$

REZ, NUM1 și NUM2 sunt valori reprezentate pe cuvânt aflate în memorie.

- a. Se va compila programul scris.
- b. Se va executa în Ollydbg
- c. Se vor urmări schimbările care au loc la nivelul regiștrilor, memoriei și a flagurilor